Python Tips and Tricks

The following topics are covered in this notebook:

1. How to Make Python Coding More Readable
2. Use Context Manager to Manage Resources
3. Use try-except-else-finally for safe exception handling
4. Use enumerate for counting
5. Use zip for parallel iteration
6. Use unpacking to assign an object to variables
7. Setattr/Getattr to dynamically set/get attributes of an object
8. Use getpass to solicit password from user
9. Use python -m to run a module as a script
10. Help/Dir for Docstings and Object Attributes
11. Use Iterable for iteration

Python Environment Staging

In [2]:
```python
import sys
print(f'Python sys.version:\n{sys.version}\n')
print(f'Python sys.executable:\n{sys.executable}\n')
print(f'Python sys.path:\n{sys.path}\n')
```

```
Python sys.version:
3.13.5 | packaged by Anaconda, Inc. | (main, Jun 12 2025, 11:23:37) [Clang 14.0.6 ]

Python sys.executable:
/Users/michaelkao/miniconda3/bin/python

Python sys.path:
['/Users/michaelkao/miniconda3/lib/python313.zip', '/Users/michaelkao/miniconda3/lib/python3.13', '/Users/michaelkao/miniconda3/lib/python3.13/lib-dynload', '', '/Users/michaelkao/min
iconda3/lib/python3.13/site-packages']
```

In [7]:
```python
import os

print(f'Current Working Dir (CWD) path:\n{os.getcwd()}\n')
```

```
Current Working Dir (CWD) path:
/Users/michaelkao/OneDrive/My Projects/Python
```

1. How to make Python code more readable

1.1 Use Descriptive Variable and Fuction Names

Choose variable and function names that describe what they store or do. For example, instead of a = 10, write user_age = 10. Clear names make your code self-explanatory and reduce the need for comments.

In [68]:
```python
# Bad
a = 5
b = 2

# Good
total = 5
sub_total = 2
print("Total:   ", total)
print("Subtotal:", sub_total)
```

```
Total:    5
Subtotal: 2
```

In [69]:
```python
# Bad
def a():
    return 5

def b():
    return 2

# Good
def calculate_total():
    return 5

def calculate_sub_total():
    return 2

print("Calling calculate_total()     =", calculate_total())
print("Calling calculate_sub_total() =", calculate_sub_total())
```

```
Calling calculate_total()     = 5
Calling calculate_sub_total() = 2
```

1.2 Use Ternary Conditionals (三元條件運算)

Python 使用三元運算子(ternary conditional operator) 好處是程式碼可以看起來比較簡短，在某些情況 python 使用三元運算子簡化後 code 會變得清爽簡短許多。

C 的寫法

```
x = condition? 1 : 0
```

Python 的寫法

```
x = 1 if condition else 0
```

二元運算的寫法較多行但邏輯相同，如下：

```
if condition:
    x = 1
else:
    x = 0
```

In [70]:
```python
condition = False

x = 1 if condition else 0

# f-strings for Formatting
# Use f-strings for cleaner, more readable string interpolation.
# They're faster and more elegant than concatenation.

print(f"condition = {condition}")
print(f"x = {x}")

print(f'condition is True') if condition else print('condition is False')
```

```
condition = False
x = 0
condition is False
```

1.3 Use _ (Underscore)for Ignored Variables or as Placeholders

When you don't need a loop variable, use _ to signal it's intentionally ignored. It's a simple but powerful convention.

Or when you don't need a loop variable, use _ to signal it's intentionally ignored. It's a simple but powerful convention.

In [71]:
```python
num1 = 10000000000
num2 = 100000000

total = num1 + num2
print(total)
```

```
10100000000
```

In [72]:
```python
# improving readability via "_" (no-op)
num1 = 10_000_000_000
num2 = 100_000_000

total = num1 + num2
print(total)

# loop counter not used
for _ in range(3):
    print("Hello")
```

```
10100000000
Hello
Hello
Hello
```

In [73]:
```python
# improving reporting via formatter
print(f'{total:,}')
```

```
10,100,000,000
```

1.4 List Comprehensions for Creating Lists

List comprehensions are a compact way to create lists. They make your code shorter and easier to understand compared to traditional loops.

In [74]:
```python
# populate list with squares of numbers from 0 to 4
squares = [x**2 for x in range(5)]
print(f"squares: {squares}")
```

```
squares: [0, 1, 4, 9, 16]
```

In [75]:
```python
# find even numbers from a list
nums = [1, 2, 3, 4, 5, 6]
evens = [num for num in nums if num % 2 == 0]
print(f"evens: {evens}")
```

```
evens: [2, 4, 6]
```

1.5 Swap two variables without using a temporary variable

In [77]:
```python
# Swapping vaiables
a = 10
b = 20
a, b = b, a
print(f"a = {a}, b = {b}")
```

```
a = 20, b = 10
```

1.6 String Processing

- Reverse a string
- Check if two strings are anagrams
- Count frequency of characters in a string
- Remove duplicates from a string

In [82]:
```python
# reverse a string
god = "god"
print(f"reversed god: {text[::-1]}")

stressed = "stressed"
print(f"reversed stressed: {stressed[::-1]}")
```

```
reversed god: dog
reversed stressed: desserts
```

In [81]:
```python
# check if two strings are anagrams
str_1 = "listen"
str_2 = "silent"

print(sorted(str_1) == sorted(str_2))
```

```
True
```

In [ ]:
```python
# count frequency of characters in a string
from collections import Counter
print(Counter([1,2,2,2,3,3,3,3]))
```

```
Counter({3: 5, 2: 3, 1: 1})
```

In [ ]:
```python
# Remove duplicates from a string
def remove_duplicates(input_str):
    # Each character will be a key in the dictionary, which eliminates duplicates
    # The values are not important, so we use None
    return ''.join(dict.fromkeys(input_str))

# Example usage
text = "hello world"
result = remove_duplicates(text)  # "helo wrd"
print(f"Original: {text}")
print(f"Result: {result}")
```

```
Original: hello world
Result: helo wrd
```

1.7 List processing

- Reverse a list
- Remove duplicates from a list
- Sort a list
- Flatten a list of lists

In [83]:
```python
# reverse the list
my_list = [1, 2, 3, 4, 5]
reversed_list = my_list[::-1]
print(f"reversed_list: {reversed_list}")

# remove duplicates from the list
my_list = [1, 2, 2, 3, 3, 4, 5]
unique_list = list(set(my_list))
print(f"unique_list: {unique_list}")

# sort the list
my_list = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
sorted_list = sorted(my_list)
print(f"sorted_list: {sorted_list}")
```

```
reversed_list: [5, 4, 3, 2, 1]
unique_list: [1, 2, 3, 4, 5]
sorted_list: [1, 1, 2, 3, 3, 4, 5, 5, 5, 6, 9]
```

In [ ]:
```python
# flatten a list of lists
lists = [[1, 2, 3], [4,5], [6, 7, 8]]
flat_list = [x for row in lists for x in row]
print(f"flat_list: {flat_list}")
```

```
flat_list: [1, 2, 3, 4, 5, 6, 7, 8]
```

1.8 Dictionary Processing

- loop through dictionary
- extract sub-dictionary

In [6]:
```python
student = {
        'name': 'John',
        'age': 25,
        'courses': ['Math', 'ComSci']
    }
# loop via items()
print("Loop via items() method:")
for key, value in student.items():
    print(f"\t{key}: {value}")

# 使用 keys() 方法遍歷
print("\nLoop via keys() method:")
for key in student.keys():
    print(f"\t{key}")

# 使用 values() 方法遍歷
print("\nLoop via values() method:")
for value in student.values():
    print(f"\t{value}")
```

```
Loop via items() method:
        name: John
        age: 25
        courses: ['Math', 'ComSci']

Loop via keys() method:
        name
        age
        courses

Loop via values() method:
        John
        25
        ['Math', 'ComSci']
```

In [7]:
```python
# Using dictionary comprehension to create sub-dictionary
original_dict = {
        'name': 'John',
        'age': 30,
        'city': 'New York',
        'occupation': 'Engineer',
        'email': 'john@example.com'
    }
print(f"original_dict: {original_dict}")

# Keys we want in the sub-dictionary
keys = ['name', 'age', 'city']
sub_dict = {key: original_dict[key] for key in keys if key in original_dict}
print(f"sub_dict: {sub_dict}")
```

```
original_dict: {'name': 'John', 'age': 30, 'city': 'New York', 'occupation': 'Engineer', 'email': 'john@example.com'}
sub_dict: {'name': 'John', 'age': 30, 'city': 'New York'}
```

2. Use Context Manager to Manage Resources

When we read from or write to a file, we sometimes forget to close it at the end. It certainly waste system resources, such as file object and buffer, once allocated and not being able to reuse.

Python has a context manager via "with" statement to manage the resouces for you once you exit the execution of "with"-bolck. Always use with open() for file operations. It ensures files are closed automatically, preventing leaks or errors.

Here is the sample file, called "test.txt", containing the following:

```
1) This is a test file
2) With multiple lines of data...
3) Third line
4) Fourth line
5) Fifth line
6) Sixth line
7) Seventh line
8) Eighth line
9) Ninth line
10) Tenth line
```

In [25]:
```python
# Open File Object Modes:
# f = open("test.txt", "r")  # for read
# f = open("test.txt", "w") # for write
# f = open("test.txt", "a") # for append
# f = open("test.txt", "r+") # for read and write

f = open('test.txt', 'r')
print(f'File object:\t {f.name}')
print(f'---> Open Mode:\t {f.mode}\n')

file_contents = f.read()
f.close()
print(f'---> is closed?\t {f.closed}\n')


print(f'read the whole file into a string variable at a time:\n{file_contents}\n')

words = file_contents.split(' ')
print(f'words in the "test.txt" file:\n{words}\n')

word_count = len(words)
print(f'There are {word_count} words in the "test.txt" file.\n')
```

```
File object:     test.txt
---> Open Mode:  r

---> is closed?  True

read the whole file into a string variable at a time:
1) This is a test file
2) With multiple lines of data...
3) Third line
4) Fourth line
5) Fifth line
6) Sixth line
7) Seventh line
8) Eighth line
9) Ninth line
10) Tenth line


words in the "test.txt" file:
['1)', 'This', 'is', 'a', 'test', 'file\n2)', 'With', 'multiple', 'lines', 'of', 'data...\n3)', 'Third', 'line\n4)', 'Fourth', 'line\n5)', 'Fifth', 'line\n6)', 'Sixth', 'line\n7)', 'S
eventh', 'line\n8)', 'Eighth', 'line\n9)', 'Ninth', 'line\n10)', 'Tenth', 'line\n']

There are 27 words in the "test.txt" file.
```

2.1 Read a File using Context Manager

The common reading methods are:

```
1. f.read()
2. f.readlines()
3. f.readline()
4. f.seek(position)
5. f.tell()
```

And attributes:

```
1. f.mode
2. f.closed
```

In [38]:
```python
# Reading Files via 'with' context manager
# not only file, but also other resources, like threads

with open('test.txt', 'r') as f:

    print(f'File object:\t {f.name}')
    print(f'---> Open Mode:\t {f.mode}\n')

    # File contents in one string variable:
    file_contents = f.read()
    print(f'The whole file was read into a string variable at a time:\n{file_contents}\n')


# Context Manager closes the file automatically once out of the with-block
print(f'---> Context Manager exits. Is the file closed?\t {f.closed}\n')

try:
    # read file when closed
    no_contents = f.read()

except ValueError:
    print('f.read(): I/O operation on closed file.\n')
except:
    print('Something else went wrong.\n')


words = file_contents.split(' ')
print(f'View words in the "test.txt" file:\n{words}\n')

word_count = len(words)
print(f'There are {word_count} words in the "test.txt" file.\n')
```

```
File object:     test.txt
---> Open Mode:  r

The whole file was read into a string variable at a time:
1) This is a test file
2) With multiple lines of data...
3) Third line
4) Fourth line
5) Fifth line
6) Sixth line
7) Seventh line
8) Eighth line
9) Ninth line
10) Tenth line


---> Context Manager exits. Is the file closed?  True

f.read(): I/O operation on closed file.

View words in the "test.txt" file:
['1)', 'This', 'is', 'a', 'test', 'file\n2)', 'With', 'multiple', 'lines', 'of', 'data...\n3)', 'Third', 'line\n4)', 'Fourth', 'line\n5)', 'Fifth', 'line\n6)', 'Sixth', 'line\n7)', 'S
eventh', 'line\n8)', 'Eighth', 'line\n9)', 'Ninth', 'line\n10)', 'Tenth', 'line\n']

There are 27 words in the "test.txt" file.

read lines into a list at a time:
['1) This is a test file\n', '2) With multiple lines of data...\n', '3) Third line\n', '4) Fourth line\n', '5) Fifth line\n', '6) Sixth line\n', '7) Seventh line\n', '8) Eighth line\
n', '9) Ninth line\n', '10) Tenth line\n']

read first line into a list at a time:
1) This is a test file

read 2nd line into a list next time:
2) With multiple lines of data...

read 3rd line into a list, ignore \n:
3) Third line
read 4th line into a list, ignore \n:
4) Fourth line
Iterating in a for-loop, line by line each loop till the end...
5) Fifth line
Iterating in a for-loop, line by line each loop till the end...
6) Sixth line
Iterating in a for-loop, line by line each loop till the end...
7) Seventh line
Iterating in a for-loop, line by line each loop till the end...
8) Eighth line
Iterating in a for-loop, line by line each loop till the end...
9) Ninth line
Iterating in a for-loop, line by line each loop till the end...
10) Tenth line

Iterating in a while-loop, 100 bytes each loop till the end...
"1) This is a test file
2) With multiple lines of data...
3) Third line
4) Fourth line
5) Fifth line
"
Iterating in a while-loop, 100 bytes each loop till the end...
"6) Sixth line
7) Seventh line
8) Eighth line
9) Ninth line
10) Tenth line
"read 10 bytes, ignore \n:
1) This is
Current position of the file objet = 10

back to BOF, read 10 bytes, ignore \n:
1) This is
Current position of the file objet = 10
```

2.2 More Examples about File Reading Methods

In [40]:
```python
# More reading methods
with open('test.txt', 'r') as f:

    # read file contents in a list of lines:
    f_contents = f.readlines()
    print(f'read lines into a list at a time:\n{f_contents}\n')

    # rewind to the head of the file
    f.seek(0)

    # read one line at a time:
    f_contents = f.readline()
    print(f'read first line into a list at a time:\n{f_contents}')

    f_contents = f.readline()
    print(f'read 2nd line into a list next time:\n{f_contents}')

    # mute "\n" effect in the buffer
    f_contents = f.readline()
    print(f'read 3rd line into a list, ignore \\n:\n{f_contents}', end='')
    f_contents = f.readline()
    print(f'read 4th line into a list, ignore \\n:\n{f_contents}', end='')

    # Iterating in a for-loop, line by line each loop till the end of file
    for line in f:
        print(f'Iterating in a for-loop, line by line each loop till the end...\n->{line}<-\n', end='')

    # rewind to the begining of file (BOF)
    f.seek(0)

    # Iterating in a while-loop in bytes till the end of file (EOF)
    size_to_read = 100
    f_contents = f.read(size_to_read)
    while len(f_contents) > 0:

        print(f'\nIterating in a while-loop, {size_to_read} bytes each loop till the end...\n->{f_contents}<-\n', end='')
        f_contents = f.read(size_to_read)

    # rewind to the head of the file
    f.seek(0)

    # read in small chunks, 10 characters per chunk:
    size_to_read = 10
    f_contents = f.read(size_to_read)
    print(f'read {size_to_read} bytes, ignore \\n:\n->{f_contents}<-\n', end='')
    print(f'Current position of the file objet = {f.tell()}\n')


    f.seek(0)
    f_contents = f.read(size_to_read)
    print(f'back to BOF, read {size_to_read} bytes, ignore \\n:\n->{f_contents}<-\n', end='')

    print(f'Current position of the file objet = {f.tell()}\n')
```

```
read lines into a list at a time:
['1) This is a test file\n', '2) With multiple lines of data...\n', '3) Third line\n', '4) Fourth line\n', '5) Fifth line\n', '6) Sixth line\n', '7) Seventh line\n', '8) Eighth line\
n', '9) Ninth line\n', '10) Tenth line\n']

read first line into a list at a time:
1) This is a test file

read 2nd line into a list next time:
2) With multiple lines of data...

read 3rd line into a list, ignore \n:
3) Third line
read 4th line into a list, ignore \n:
4) Fourth line
Iterating in a for-loop, line by line each loop till the end...
->5) Fifth line
<-
Iterating in a for-loop, line by line each loop till the end...
->6) Sixth line
<-
Iterating in a for-loop, line by line each loop till the end...
->7) Seventh line
<-
Iterating in a for-loop, line by line each loop till the end...
->8) Eighth line
<-
Iterating in a for-loop, line by line each loop till the end...
->9) Ninth line
<-
Iterating in a for-loop, line by line each loop till the end...
->10) Tenth line
<-

Iterating in a while-loop, 100 bytes each loop till the end...
->1) This is a test file
2) With multiple lines of data...
3) Third line
4) Fourth line
5) Fifth line
<-

Iterating in a while-loop, 100 bytes each loop till the end...
->6) Sixth line
7) Seventh line
8) Eighth line
9) Ninth line
10) Tenth line
<-
read 10 bytes, ignore \n:
->1) This is<-
Current position of the file objet = 10

back to BOF, read 10 bytes, ignore \n:
->1) This is<-
Current position of the file objet = 10
```

2.3 Write a File using Context Manager

In [43]:
```python
# Write to a file using Context Manager
with open("test2.txt", "w") as f:
    f.write("Testing123")
    f.seek(0)
    f.write("Test987\n")

with open("test2.txt", "r") as f:
    # Iterating line by line till the end of file:
    for line in f:
        print(f'Iterating line by line till the end...\n->{line}<-\n', end='')
```

```
Iterating line by line till the end...
->Test987
<-
Iterating line by line till the end...
->23<-
```

2.4 Copy a File using Context Manager

In [1]:
```python
# Copy a File to Another:
with open("test.txt", "r") as rf:
    with open("test_copy.txt", "w") as wf:
        for line in rf:
            wf.write(line)

file_read = "test_copy.txt"
with open(file_read, "r") as rf:
    print(f'opening {file_read} file:\n')
    for line in rf:
        print(f'Iterating line by line till the end...\n->{line}<-\n', end='')


# Copy an image: read/write in binary
with open("Olisan.JPG", "rb") as rf:
    with open("Olisan_copy.JPG", "wb") as wf:
        for line in rf:
            wf.write(line)

# Copy an image: in chunks
with open("Olisan.JPG", "rb") as rf:
    with open("Olisan_copy2.JPG", "wb") as wf:
        chunk_size = 4096
        rf_chunk = rf.read(chunk_size)

        while len(rf_chunk) > 0:
            wf.write(rf_chunk)
            rf_chunk = rf.read(chunk_size)

# Show all the images
from PIL import Image
img = Image.open('Olisan.JPG')
img.show()
img_copy = Image.open('Olisan_copy.JPG')
img_copy.show()
img_copy2 = Image.open('Olisan_copy2.JPG')
img_copy2.show()
```

```
opening test_copy.txt file:

Iterating line by line till the end...
->1) This is a test file
<-
Iterating line by line till the end...
->2) With multiple lines of data...
<-
Iterating line by line till the end...
->3) Third line
<-
Iterating line by line till the end...
->4) Fourth line
<-
Iterating line by line till the end...
->5) Fifth line
<-
Iterating line by line till the end...
->6) Sixth line
<-
Iterating line by line till the end...
->7) Seventh line
<-
Iterating line by line till the end...
->8) Eighth line
<-
Iterating line by line till the end...
->9) Ninth line
<-
Iterating line by line till the end...
->10) Tenth line
<-
```

3. Use try-except-else-finally for error handling

Use try/except to handle runtime errors gracefully. It prevents your code from crashing unexpectedly.

In [9]:
```python
try:
    # Code that might raise an exception
    result = 10 / 0
except ZeroDivisionError:
    # Handle the specific exception
    print("Error: Cannot divide by zero")
else:
    # Code to run if no exception occurs
    print("else: Division successful")
finally:
    # Code to run regardless of whether an exception occurred or not
    print("finally: This will always execute")
```

```
Error: Cannot divide by zero
finally: This will always execute
```

4. Use enumerate for counting

When looping through lists, use enumerate() instead of manual counters. It automatically gives you both index and value, improving clarity.

In [3]:
```python
# Old way using counter
names = ['Corey', 'Michael', 'Travis', 'Dave']

index = 0
for name in names:
    print(index, name)
    index += 1
```

```
0 Corey
1 Michael
2 Travis
3 Dave
```

In [4]:
```python
# cleaner code with enumerator
names = ['Corey', 'Michael', 'Travis', 'Dave']

for index, name in enumerate(names):
    print(index, name)
```

```
0 Corey
1 Michael
2 Travis
3 Dave
```

In [5]:
```python
# Another cleaner code with enumerator, starting index from 1
names = ['Corey', 'Michael', 'Travis', 'Dave']

# changing starting index from 1
for index, name in enumerate(names, start=1):
    print(index, name)
```

```
1 Corey
2 Michael
3 Travis
4 Dave
```

5. Use zip for multiple objects

Combine multiple lists cleanly with zip(). It's perfect for looping through related data side by side.

In [13]:
```python
names = ['Corey', 'Michael', 'Travis', 'Dave']
heroes = ['Spiderman', 'Superman', 'Deadpool', 'Batman']
for index, name in enumerate(names):
    hero = heroes[index]
    print(f'{name} is actually {hero}')
```

```
Corey is actually Spiderman
Michael is actually Superman
Travis is actually Deadpool
Dave is actually Batman
```

In [15]:
```python
names = ['Corey', 'Michael', 'Travis', 'Dave']
heroes = ['Spiderman', 'Superman', 'Deadpool', 'Batman']

# cleaner code with zip
for name, hero in zip(names, heroes):
    print(f'{name} is actually {hero}')
```

```
Corey is actually Spiderman
Michael is actually Superman
Travis is actually Deadpool
Dave is actually Batman
```

In [16]:
```python
names = ['Corey', 'Michael', 'Travis', 'Dave']
heroes = ['Spiderman', 'Superman', 'Deadpool', 'Batman']
universes = ['Marvel', 'DC', 'Tokyo', 'Taiwan']

# cleaner code with zip more than 2 lists
for name, hero, universe in zip(names, heroes, universes):
    print(f'{name} is actually {hero} from {universe}')
```

```
Corey is actually Spiderman from Marvel
Michael is actually Superman from DC
Travis is actually Deadpool from Tokyo
Dave is actually Batman from Taiwan
```

6. Use unpacking to assign an object to variables

It allows you to assign values to variables in a single line of code.

It allows you to assign values to variables in a single line of code.

In [13]:
```python
# Basic unpacking with tuple
a, b = (1, 2)
print(f"a: {a}")
print(f"b: {b}")
```

```
a: 1
b: 2
```

In [14]:
```python
# unpacking a list
x, y = [10, 20]
print(f"x: {x}, y: {y}")
```

```
x: 10, y: 20
```

In [16]:
```python
# Unpack in function arguments
def add(a, b, c):
    return a + b + c

args = (1, 2, 3)
print(f"add(*args) = {add(*args)}")
```

```
add(*args) = 6
```

In [17]:
```python
# unpack a string which is an iterable, consisting a sequence of characters
a, b, c = "abc"
print(f"a={a}, b={b}, c={c}")
```

```
a=a, b=b, c=c
```

In [18]:
```python
# Unpacking a dictionary, using ** to unpack key-value pairs
def greet(name, age):
    print(f"Hello, {name}. You are {age} years old.")

person = {"name": "John", "age": 30}
greet(**person)

# same as
greet(name="John", age=30)
```

```
Hello, John. You are 30 years old.
Hello, John. You are 30 years old.
```

In [19]:
```python
# unpacking keys or valures from a dictionary
d = {'a': 1, 'b': 2, 'c': 3}

k1, k2, k3 = d.keys()
v1, v2, v3 = d.values()

print(f"k1={k1}, k2={k2}, k3={k3}")
print(f"v1={v1}, v2={v2}, v3={v3}")
```

```
k1=a, k2=b, k3=c
v1=1, v2=2, v3=3
```

In [11]:
```python
# Use * for extended unpacking (Python3+)
a, b, *c, d = (1, 2, 3, 4, 5, 6)
print(a)
print(b)
print(c)
print(d)
```

```
1
2
[3, 4, 5]
6
```

In [10]:
```python
a, b, *_, d = (1, 2, 3, 4, 5, 6)
print(a)
print(b)
print(d)
```

```
1
2
6
```

In [21]:
```python
# nested unpacking
coords =[(1,2),(3,4),(5,6)]
x,y = zip(*coords)
print(f"x: {x}")
print(f"y: {y}")
```

```
x: (1, 3, 5)
y: (2, 4, 6)
```

In [22]:
```python
# more complex nested list
data = ['Michael', (25, 'Engineer')]

# unpacking
name, (age, profession) = data
print(f"name: {name}, age: {age}, profession: {profession}")
```

```
name: Michael, age: 25, profession: Engineer
```

7. Setattr/Getattr to dynamically set/get attributes of an object

Dynamically setting and getting attributes of an object.

In [2]:
```python
class Person():
    pass

person = Person()

# setting attributes and values on the fly
person.first = "Michael"
person.last = "Kao"

print(f'{person.first} {person.last}')
```

```
Michael Kao
```

In [3]:
```python
class Person():
    pass

person = Person()

# setting an attribute with the value of a variable on the fly
key = "first"
value = "Michael"
person.last = "Kao"

setattr(person, key, value)

print(f'{person.first} {person.last}')
```

```
Michael Kao
```

In [4]:
```python
class Person():
    pass

person = Person()

# setting an attribute by the value of a variable on the fly
key = "first"
value = "Michael"
person.last = "Kao"

setattr(person, key, value)

# getting an attribute value by the value of a variable
first = getattr(person, key)
print(first)
```

```
Michael
```

In [6]:
```python
class Person():
    pass

person = Person()

person_dict = {'first':'Michael', 'last':'Kao'}

for key, value in person_dict.items():
    setattr(person, key, value)

print(f'{person.first} {person.last}')

for key in person_dict.keys():
    print(getattr(person, key))
```

```
Michael Kao
Michael
Kao
```

8. Use getpass to solicit password from user

Solicitate a password from user, and not showing it on terminal

In [7]:
```python
username = input('Username: ')
password = input('Password: ')

print('Logging In...')
```

```
Username: Michael
Password: Kao
Logging In...
```

In [8]:
```python
from getpass import getpass

username = input('Username: ')
password = getpass('Password: ')

print('Logging In...')
```

```
Username: Michael
Password: ········
Logging In...
```

9. Python dash m

Spawn a process at the cmd prompt with args

```
python -m module_name
```

In [9]:
```python
# Run a command line from Jupyter notebook cell
!python -m pip list
```

```
Package                   Version
------------------------- --------------
absl-py                   2.1.0
annotated-doc             0.0.3
annotated-types           0.7.0
anyio                     4.9.0
appdirs                   1.4.4
appnope                   0.1.4
argon2-cffi               25.1.0
argon2-cffi-bindings      21.2.0
arrow                     1.3.0
asttokens                 3.0.0
async-lru                 2.0.5
attrs                     25.3.0
babel                     2.17.0
beautifulsoup4            4.13.4
bleach                    6.2.0
bs4                       0.0.2
certifi                   2025.4.26
cffi                      1.17.1
charset-normalizer        3.4.0
click                     8.1.8
comm                      0.2.2
contourpy                 1.3.1
cycler                    0.12.1
debugpy                   1.8.11
decorator                 5.2.1
defusedxml                0.7.1
dnspython                 2.7.0
email_validator           2.2.0
et_xmlfile                2.0.0
exceptiongroup            1.3.0
executing                 2.2.0
fastapi                   0.121.0
fastjsonschema            2.21.1
fibheap                   0.2.1
fonttools                 4.57.0
fqdn                      1.5.1
glog                      0.3.1
graphviz                  0.20.3
greenlet                  3.2.3
h11                       0.16.0
httpcore                  1.0.9
httpx                     0.28.1
idna                      3.10
importlib_metadata        8.7.0
ipykernel                 6.29.5
ipython                   9.2.0
ipython_pygments_lexers   1.1.1
isoduration               20.11.0
jedi                      0.19.2
Jinja2                    3.1.6
json5                     0.12.0
jsonpointer               3.0.0
jsonschema                4.24.0
jsonschema-specifications 2025.4.1
jupyter_client            8.6.3
jupyter_core              5.9.1
jupyter-events            0.12.0
jupyter-lsp               2.2.5
jupyter_server            2.16.0
jupyter_server_terminals  0.5.3
jupyterlab                4.4.10
jupyterlab_pygments       0.3.0
jupyterlab_server         2.27.3
kiwisolver                1.4.8
MarkupSafe                3.0.2
matplotlib                3.10.1
matplotlib-inline         0.1.7
mistune                   3.1.3
MouseInfo                 0.1.3
nbclient                  0.10.2
nbconvert                 7.16.6
nbformat                  5.10.4
nest_asyncio              1.6.0
notebook                  7.4.3
notebook_shim             0.2.4
numpy                     2.1.3
openpyxl                  3.1.5
overrides                 7.7.0
packaging                 24.2
pandas                    2.2.3
pandoc                    2.4
pandocfilters             1.5.1
parso                     0.8.4
pexpect                   4.9.0
pickleshare               0.7.5
pillow                    11.1.0
pip                       25.3
platformdirs              4.3.8
playwright                1.52.0
plumbum                   1.9.0
ply                       3.11
prometheus_client         0.22.1
prompt_toolkit            3.0.51
psutil                    7.0.0
ptyprocess                0.7.0
pure_eval                 0.2.3
PyAutoGUI                 0.9.54
pycparser                 2.22
pydantic                  2.11.3
pydantic_core             2.33.1
pyee                      13.0.0
pygame                    2.6.1
PyGetWindow               0.0.9
Pygments                  2.19.1
PyMsgBox                  1.0.9
pyobjc-core               11.0
pyobjc-framework-Cocoa    11.0
pyobjc-framework-Quartz   11.0
pyparsing                 3.2.3
pyperclip                 1.9.0
pyppeteer                 2.0.0
PyQt5                     5.15.11
PyQt5-Qt5                 5.15.17
PyQt5_sip                 12.17.0
PyRect                    0.2.0
PyScreeze                 1.0.1
python-dateutil           2.9.0.post0
python-dotenv             1.1.0
python-gflags             3.1.2
python-json-logger        3.3.0
pytweening                1.2.0
pytz                      2024.2
PyYAML                    6.0.2
pyzmq                     26.2.0
qrcode                    8.1
referencing               0.36.2
requests                  2.32.5
rfc3339-validator         0.1.4
rfc3986-validator         0.1.1
rpds-py                   0.25.1
rubicon-objc              0.5.0
schedule                  1.2.2
Send2Trash                1.8.3
setuptools                80.9.0
six                       1.16.0
sniffio                   1.3.1
sortedcollections         2.1.0
sortedcontainers          2.4.0
soupsieve                 2.7
stack_data                0.6.3
starlette                 0.49.3
terminado                 0.18.1
tinycss2                  1.4.0
tornado                   6.5
tqdm                      4.67.1
traitlets                 5.14.3
types-python-dateutil     2.9.0.20250516
typing_extensions         4.13.2
typing-inspection         0.4.0
tzdata                    2024.2
uri-template              1.3.0
urllib3                   2.5.0
uv                        0.9.7
uvicorn                   0.34.1
wcwidth                   0.2.13
webcolors                 24.11.1
webencodings              0.5.1
websocket-client          1.8.0
websockets                10.4
wheel                     0.44.0
xlrd                      2.0.1
zipp                      3.21.0
```

10. Help/Dir for Docstings and Object Attributes

In [3]:
```python
import uvicorn

# query online document
help(uvicorn)
```

```
Help on package uvicorn:

NAME
    uvicorn

PACKAGE CONTENTS
    __main__
    _subprocess
    _types
    config
    importer
    lifespan (package)
    logging
    loops (package)
    main
    middleware (package)
    protocols (package)
    server
    supervisors (package)
    workers

CLASSES
    builtins.object
        uvicorn.config.Config
        uvicorn.server.Server

    class Config(builtins.object)
     |  Config(
     |      app: 'ASGIApplication | Callable[..., Any] | str',
     |      host: 'str' = '127.0.0.1',
     |      port: 'int' = 8000,
     |      uds: 'str | None' = None,
     |      fd: 'int | None' = None,
     |      loop: 'LoopSetupType' = 'auto',
     |      http: 'type[asyncio.Protocol] | HTTPProtocolType' = 'auto',
     |      ws: 'type[asyncio.Protocol] | WSProtocolType' = 'auto',
     |      ws_max_size: 'int' = 16777216,
     |      ws_max_queue: 'int' = 32,
     |      ws_ping_interval: 'float | None' = 20.0,
     |      ws_ping_timeout: 'float | None' = 20.0,
     |      ws_per_message_deflate: 'bool' = True,
     |      lifespan: 'LifespanType' = 'auto',
     |      env_file: 'str | os.PathLike[str] | None' = None,
     |      log_config: 'dict[str, Any] | str | RawConfigParser | IO[Any] | None' = {'version': 1, 'disable_existing_loggers': False, 'formatters': {'default': {'()': 'uvicorn.loggin
g.DefaultFormatter', 'fmt': '%(levelprefix)s %(message)s', 'use_colors': None}, 'access': {'()': 'uvicorn.logging.AccessFormatter', 'fmt': '%(levelprefix)s %(client_addr)s - "%(reques
t_line)s" %(status_code)s'}}, 'handlers': {'default': {'formatter': 'default', 'class': 'logging.StreamHandler', 'stream': 'ext://sys.stderr'}, 'access': {'formatter': 'access', 'clas
s': 'logging.StreamHandler', 'stream': 'ext://sys.stdout'}}, 'loggers': {'uvicorn': {'handlers': ['default'], 'level': 'INFO', 'propagate': False}, 'uvicorn.error': {'level': 'INFO'},
'uvicorn.access': {'handlers': ['access'], 'level': 'INFO', 'propagate': False}}},
     |      log_level: 'str | int | None' = None,
     |      access_log: 'bool' = True,
     |      use_colors: 'bool | None' = None,
     |      interface: 'InterfaceType' = 'auto',
     |      reload: 'bool' = False,
     |      reload_dirs: 'list[str] | str | None' = None,
     |      reload_delay: 'float' = 0.25,
     |      reload_includes: 'list[str] | str | None' = None,
     |      reload_excludes: 'list[str] | str | None' = None,
     |      workers: 'int | None' = None,
     |      proxy_headers: 'bool' = True,
     |      server_header: 'bool' = True,
     |      date_header: 'bool' = True,
     |      forwarded_allow_ips: 'list[str] | str | None' = None,
     |      root_path: 'str' = '',
     |      limit_concurrency: 'int | None' = None,
     |      limit_max_requests: 'int | None' = None,
     |      backlog: 'int' = 2048,
     |      timeout_keep_alive: 'int' = 5,
     |      timeout_notify: 'int' = 30,
     |      timeout_graceful_shutdown: 'int | None' = None,
     |      callback_notify: 'Callable[..., Awaitable[None]] | None' = None,
     |      ssl_keyfile: 'str | os.PathLike[str] | None' = None,
     |      ssl_certfile: 'str | os.PathLike[str] | None' = None,
     |      ssl_keyfile_password: 'str | None' = None,
     |      ssl_version: 'int' = <_SSLMethod.PROTOCOL_TLS_SERVER: 17>,
     |      ssl_cert_reqs: 'int' = <VerifyMode.CERT_NONE: 0>,
     |      ssl_ca_certs: 'str | None' = None,
     |      ssl_ciphers: 'str' = 'TLSv1',
     |      headers: 'list[tuple[str, str]] | None' = None,
     |      factory: 'bool' = False,
     |      h11_max_incomplete_event_size: 'int | None' = None
     |  )
     |
     |  Methods defined here:
     |
     |  __init__(
     |      self,
     |      app: 'ASGIApplication | Callable[..., Any] | str',
     |      host: 'str' = '127.0.0.1',
     |      port: 'int' = 8000,
     |      uds: 'str | None' = None,
     |      fd: 'int | None' = None,
     |      loop: 'LoopSetupType' = 'auto',
     |      http: 'type[asyncio.Protocol] | HTTPProtocolType' = 'auto',
     |      ws: 'type[asyncio.Protocol] | WSProtocolType' = 'auto',
     |      ws_max_size: 'int' = 16777216,
     |      ws_max_queue: 'int' = 32,
     |      ws_ping_interval: 'float | None' = 20.0,
     |      ws_ping_timeout: 'float | None' = 20.0,
     |      ws_per_message_deflate: 'bool' = True,
     |      lifespan: 'LifespanType' = 'auto',
     |      env_file: 'str | os.PathLike[str] | None' = None,
     |      log_config: 'dict[str, Any] | str | RawConfigParser | IO[Any] | None' = {'version': 1, 'disable_existing_loggers': False, 'formatters': {'default': {'()': 'uvicorn.loggin
g.DefaultFormatter', 'fmt': '%(levelprefix)s %(message)s', 'use_colors': None}, 'access': {'()': 'uvicorn.logging.AccessFormatter', 'fmt': '%(levelprefix)s %(client_addr)s - "%(reques
t_line)s" %(status_code)s'}}, 'handlers': {'default': {'formatter': 'default', 'class': 'logging.StreamHandler', 'stream': 'ext://sys.stderr'}, 'access': {'formatter': 'access', 'clas
s': 'logging.StreamHandler', 'stream': 'ext://sys.stdout'}}, 'loggers': {'uvicorn': {'handlers': ['default'], 'level': 'INFO', 'propagate': False}, 'uvicorn.error': {'level': 'INFO'},
'uvicorn.access': {'handlers': ['access'], 'level': 'INFO', 'propagate': False}}},
     |      log_level: 'str | int | None' = None,
     |      access_log: 'bool' = True,
     |      use_colors: 'bool | None' = None,
     |      interface: 'InterfaceType' = 'auto',
     |      reload: 'bool' = False,
     |      reload_dirs: 'list[str] | str | None' = None,
     |      reload_delay: 'float' = 0.25,
     |      reload_includes: 'list[str] | str | None' = None,
     |      reload_excludes: 'list[str] | str | None' = None,
     |      workers: 'int | None' = None,
     |      proxy_headers: 'bool' = True,
     |      server_header: 'bool' = True,
     |      date_header: 'bool' = True,
     |      forwarded_allow_ips: 'list[str] | str | None' = None,
     |      root_path: 'str' = '',
     |      limit_concurrency: 'int | None' = None,
     |      limit_max_requests: 'int | None' = None,
     |      backlog: 'int' = 2048,
     |      timeout_keep_alive: 'int' = 5,
     |      timeout_notify: 'int' = 30,
     |      timeout_graceful_shutdown: 'int | None' = None,
     |      callback_notify: 'Callable[..., Awaitable[None]] | None' = None,
     |      ssl_keyfile: 'str | os.PathLike[str] | None' = None,
     |      ssl_certfile: 'str | os.PathLike[str] | None' = None,
     |      ssl_keyfile_password: 'str | None' = None,
     |      ssl_version: 'int' = <_SSLMethod.PROTOCOL_TLS_SERVER: 17>,
     |      ssl_cert_reqs: 'int' = <VerifyMode.CERT_NONE: 0>,
     |      ssl_ca_certs: 'str | None' = None,
     |      ssl_ciphers: 'str' = 'TLSv1',
     |      headers: 'list[tuple[str, str]] | None' = None,
     |      factory: 'bool' = False,
     |      h11_max_incomplete_event_size: 'int | None' = None
     |  )
     |      Initialize self.  See help(type(self)) for accurate signature.
     |
     |  bind_socket(self) -> 'socket.socket'
     |
     |  configure_logging(self) -> 'None'
     |
     |  load(self) -> 'None'
     |
     |  setup_event_loop(self) -> 'None'
     |
     |  ----------------------------------------------------------------------
     |  Readonly properties defined here:
     |
     |  asgi_version
     |
     |  is_ssl
     |
     |  should_reload
     |
     |  use_subprocess
     |
     |  ----------------------------------------------------------------------
     |  Data descriptors defined here:
     |
     |  __dict__
     |      dictionary for instance variables
     |
     |  __weakref__
     |      list of weak references to the object

    class Server(builtins.object)
     |  Server(config: 'Config') -> 'None'
     |
     |  Methods defined here:
     |
     |  __init__(self, config: 'Config') -> 'None'
     |      Initialize self.  See help(type(self)) for accurate signature.
     |
     |  capture_signals(self) -> 'Generator[None, None, None]'
     |
     |  handle_exit(self, sig: 'int', frame: 'FrameType | None') -> 'None'
     |
     |  async main_loop(self) -> 'None'
     |
     |  async on_tick(self, counter: 'int') -> 'bool'
     |
     |  run(self, sockets: 'list[socket.socket] | None' = None) -> 'None'
     |
     |  async serve(self, sockets: 'list[socket.socket] | None' = None) -> 'None'
     |
     |  async shutdown(self, sockets: 'list[socket.socket] | None' = None) -> 'None'
     |
     |  async startup(self, sockets: 'list[socket.socket] | None' = None) -> 'None'
     |
     |  ----------------------------------------------------------------------
     |  Data descriptors defined here:
     |
     |  __dict__
     |      dictionary for instance variables
     |
     |  __weakref__
     |      list of weak references to the object

FUNCTIONS
    run(
        app: 'ASGIApplication | Callable[..., Any] | str',
        *,
        host: 'str' = '127.0.0.1',
        port: 'int' = 8000,
        uds: 'str | None' = None,
        fd: 'int | None' = None,
        loop: 'LoopSetupType' = 'auto',
        http: 'type[asyncio.Protocol] | HTTPProtocolType' = 'auto',
        ws: 'type[asyncio.Protocol] | WSProtocolType' = 'auto',
        ws_max_size: 'int' = 16777216,
        ws_max_queue: 'int' = 32,
        ws_ping_interval: 'float | None' = 20.0,
        ws_ping_timeout: 'float | None' = 20.0,
        ws_per_message_deflate: 'bool' = True,
        lifespan: 'LifespanType' = 'auto',
        interface: 'InterfaceType' = 'auto',
        reload: 'bool' = False,
        reload_dirs: 'list[str] | str | None' = None,
        reload_includes: 'list[str] | str | None' = None,
        reload_excludes: 'list[str] | str | None' = None,
        reload_delay: 'float' = 0.25,
        workers: 'int | None' = None,
        env_file: 'str | os.PathLike[str] | None' = None,
        log_config: 'dict[str, Any] | str | RawConfigParser | IO[Any] | None' = {'version': 1, 'disable_existing_loggers': False, 'formatters': {'default': {'()': 'uvicorn.logging.Def
aultFormatter', 'fmt': '%(levelprefix)s %(message)s', 'use_colors': None}, 'access': {'()': 'uvicorn.logging.AccessFormatter', 'fmt': '%(levelprefix)s %(client_addr)s - "%(request_lin
e)s" %(status_code)s'}}, 'handlers': {'default': {'formatter': 'default', 'class': 'logging.StreamHandler', 'stream': 'ext://sys.stderr'}, 'access': {'formatter': 'access', 'class': '
logging.StreamHandler', 'stream': 'ext://sys.stdout'}}, 'loggers': {'uvicorn': {'handlers': ['default'], 'level': 'INFO', 'propagate': False}, 'uvicorn.error': {'level': 'INFO'}, 'uvi
corn.access': {'handlers': ['access'], 'level': 'INFO', 'propagate': False}}},
        log_level: 'str | int | None' = None,
        access_log: 'bool' = True,
        proxy_headers: 'bool' = True,
        server_header: 'bool' = True,
        date_header: 'bool' = True,
        forwarded_allow_ips: 'list[str] | str | None' = None,
        root_path: 'str' = '',
        limit_concurrency: 'int | None' = None,
        backlog: 'int' = 2048,
        limit_max_requests: 'int | None' = None,
        timeout_keep_alive: 'int' = 5,
        timeout_graceful_shutdown: 'int | None' = None,
        ssl_keyfile: 'str | os.PathLike[str] | None' = None,
        ssl_certfile: 'str | os.PathLike[str] | None' = None,
        ssl_keyfile_password: 'str | None' = None,
        ssl_version: 'int' = <_SSLMethod.PROTOCOL_TLS_SERVER: 17>,
        ssl_cert_reqs: 'int' = <VerifyMode.CERT_NONE: 0>,
        ssl_ca_certs: 'str | None' = None,
        ssl_ciphers: 'str' = 'TLSv1',
        headers: 'list[tuple[str, str]] | None' = None,
        use_colors: 'bool | None' = None,
        app_dir: 'str | None' = None,
        factory: 'bool' = False,
        h11_max_incomplete_event_size: 'int | None' = None
    ) -> 'None'

DATA
    __all__ = ['main', 'run', 'Config', 'Server']
    main = <Command main>

VERSION
    0.34.1

FILE
    /Users/michaelkao/miniconda3/envs/pex/lib/python3.13/site-packages/uvicorn/__init__.py
```

In [5]:
```python
import uvicorn
# query object info
dir(uvicorn)
```

Out[5]:
```
['Config',
 'Server',
 '__all__',
 '__builtins__',
 '__cached__',
 '__doc__',
 '__file__',
 '__loader__',
 '__name__',
 '__package__',
 '__path__',
 '__spec__',
 '__version__',
 '_subprocess',
 '_types',
 'config',
 'importer',
 'logging',
 'main',
 'middleware',
 'run',
 'server',
 'supervisors']
```

In [6]:
```python
import uvicorn
# simply use a "?" after an object to find the associated file directly
uvicorn?
```

```
Type:        module
String form: <module 'uvicorn' from '/Users/michaelkao/miniconda3/envs/pex/lib/python3.13/site-packages/uvicorn/__init__.py'>
File:        ~/miniconda3/envs/pex/lib/python3.13/site-packages/uvicorn/__init__.py
Docstring:   <no docstring>
```

In [7]:
```python
from datetime import datetime

# finding attributes and methods of an function object
dir(datetime)
```

Out[7]:
```
['__add__',
 '__class__',
 '__delattr__',
 '__dir__',
 '__doc__',
 '__eq__',
 '__format__',
 '__ge__',
 '__getattribute__',
 '__getstate__',
 '__gt__',
 '__hash__',
 '__init__',
 '__init_subclass__',
 '__le__',
 '__lt__',
 '__ne__',
 '__new__',
 '__radd__',
 '__reduce__',
 '__reduce_ex__',
 '__replace__',
 '__repr__',
 '__rsub__',
 '__setattr__',
 '__sizeof__',
 '__str__',
 '__sub__',
 '__subclasshook__',
 'astimezone',
 'combine',
 'ctime',
 'date',
 'day',
 'dst',
 'fold',
 'fromisocalendar',
 'fromisoformat',
 'fromordinal',
 'fromtimestamp',
 'hour',
 'isocalendar',
 'isoformat',
 'isoweekday',
 'max',
 'microsecond',
 'min',
 'minute',
 'month',
 'now',
 'replace',
 'resolution',
 'second',
 'strftime',
 'strptime',
 'time',
 'timestamp',
 'timetuple',
 'timetz',
 'today',
 'toordinal',
 'tzinfo',
 'tzname',
 'utcfromtimestamp',
 'utcnow',
 'utcoffset',
 'utctimetuple',
 'weekday',
 'year']
```

In [8]:
```python
from datetime import datetime

# what is it?
print(datetime.today)

# running it
print(datetime.today())
```

```
<built-in method today of type object at 0x1036f48b0>
2025-11-05 14:39:42.333854
```

11. Iterables for iteration

Iterable is an object, that one can iterate over. It generates an Iterator when passed to iter() method.

An iterator is an object, which is used to iterate over an iterable object using the `__next__()` method.

Iterators have the `__next__()` method, which returns the next item of the object. Note that every iterator is also an iterable, but not every iterable is an iterator.

For example, a list is iterable but a list is not an iterator. An iterator can be created from an iterable by using the function iter(). To make this possible, the class of an object needs either a method `__iter__`, which returns an iterator, or a `__getitem__` method with sequential indexes starting with 0.

In [ ]:
```python
for element in ['c123', 142_857, 'Michael', 'Kao']:
    print(element, end=" ")
print()

for element in ('c123', 142_857, 'Michael', 'Kao'):
    print(element, end=" ")
print()

for letter in 'Michael Kao':
    print(letter, end=" ")
print()

for byte in b'Binary':
    print(byte, end=" ")
print()

# All objects, shown above, are iterables
# However, the integer type is not literable, as follows
c = 142_857

# must convert to a string to make  iteger iterable
for digit in str(c):
    print(digit, end=" ")
print()

# or convert to a list to make  iteger iterable
digits = [int(d) for d in str(c)]
for digit in digits:
    print(digit, end=" ")
print()

# make a deck of cards iterable?
import itertools
ranks = list(range(2, 11)) + ["J", "Q", "K", "A"]
ranks = [str(rank) for rank in ranks]

print(f'Ranks: {ranks}')

suits = ["Spades", "Hearts", "Dimonds", "Clubs"]
deck = [card for card in itertools.product(suits, ranks)]

for (index, card) in enumerate(deck):
    print(f'{1+index}: {card}')

hands = [hand for hand in itertools.combinations(deck, 5)]
print(f'The number of 5-card poker has {len(hands):,} combinations.')
```